

A type trait for `std::compare_3way()`'s type

Contents

1. [Motivation](#)
2. [Proposal](#)
 - 2.1. [Wording](#)
 - 2.2. [Interaction with P1186](#)
3. [References](#)

§ 1. Motivation

For some types, in order to implement `operator<=>()` you have to defer to the implementation of the comparison operators of other types. And more than that, you need to know what the comparison category is for those types you defer to in order to know what comparison category to return. When I went through the exercise of trying to [implement `operator<=>` for `optional<T>`](#), the declarations of the three functions ended up looking like this (rewritten here as non-member operator templates just for clarity):

```
template <typename T, typename U>
constexpr auto operator<=>(optional<T> const& lhs, optional<U> const& rhs)
    -> decltype(compare_3way(*lhs, *rhs));

template <typename T, typename U>
constexpr auto operator<=>(optional<T> const& lhs, U const& rhs)
    -> decltype(compare_3way(*lhs, rhs));

template <typename T>
constexpr auto operator<=>(optional<T> const&, nullopt_t)
    -> strong_ordering;
```

We need to use `std::compare_3way(*lhs, *rhs)` instead of writing `lhs <=> rhs` to also handle those types which do not yet implement `<=>`, and that function is [specified](#) in a way to correctly address all the possible situations.

Let's throw in a few more examples for implementations for `vector` and `expected`:

```
template <typename T, typename U>
auto operator<=>(vector<T> const& lhs, vector<U> const& rhs)
    -> decltype(compare_3way(lhs[0], rhs[0]));

template <typename T1, typename E1, typename T2, typename E2>
auto operator<=>(expected<T1,E1> const& lhs, expected<T2,E2> const& rhs)
    -> common_comparison_category_t<
        decltype(compare_3way(lhs.value(), rhs.value())),
        decltype(compare_3way(lhs.error(), rhs.error()))>;
```

We see the same thing over and over and over again. We need to know what comparison category to return, and this comparison category is a property of the types that we're comparing.

That's a type trait. A type trait that's currently missing from the standard library.

§ 2. Proposal

This paper proposes the addition of a new type trait based on the preexisting rules in `[alg.3way]` and respecifying `std::compare_3way()` to use that trait in its return type instead of `auto`. This mirrors the usage of `std::invoke_result_t` as the return type of `std::invoke()`, and so should be named either `compare_3way_result` or `compare_3way_type`.

The trait should be a binary type trait, since ultimately we're comparing two things, but for convenience for common cases, the second type parameter should be defaulted to the first, so that users can simply write `compare_3way_type<T>` instead of `compare_3way_type<T,T>`.

Because comparison should not modify its arguments and `compare_3way()` takes its arguments by reference to const anyway, `compare_3way_type` is specified in such a way as to not require adding `const&` to every argument all the time.

The trait would allow for less cumbersome declarations of all of these operator templates:

Today	Proposed
<pre>template <typename T, typename U> constexpr auto operator<=>(optional<T> const& lhs, optional<U> const& rhs) -> decltype(compare_3way(*lhs, *rhs));</pre>	<pre>template <typename T, typename U> constexpr auto operator<=>(optional<T> const& lhs, optional<U> const& rhs) -> compare_3way_type_t<T, U>;</pre>
<pre>template <typename T, typename U> auto operator<=>(vector<T> const& lhs, vector<U> const& rhs) -> decltype(compare_3way(lhs[0], rhs[0]));</pre>	<pre>template <typename T, typename U> auto operator<=>(vector<T> const& lhs, vector<U> const& rhs) -> compare_3way_type_t<T, U>;</pre>
<pre>template <typename T1, typename E1, typename T2, typename E2> auto operator<=>(expected<T1,E1> const& lhs, expected<T2,E2> const& rhs) -> common_comparison_category_t< decltype(compare_3way(lhs.value(), rhs.value())), decltype(compare_3way(lhs.error(), rhs.error()))>;</pre>	<pre>template <typename T1, typename E1, typename T2, typename E2> auto operator<=>(expected<T1,E1> const& lhs, expected<T2,E2> const& rhs) -> common_comparison_category_t< compare_3way_type_t<T1, T2>, compare_3way_type_t<E1, E2>>;</pre>

§ 2.1. Wording

Add the new trait and its use into the `<algorithm>` synopsis in 23.4 [algorithm.sym]:

```
#include <initializer_list>

namespace std {
    // ...

    // [alg.3way], three-way comparison algorithms
```

```

template<class T, class U = T> struct compare_3way_type;

template<class T, class U = T>
using compare_3way_type_t = typename compare_3way_type<T, U>::type;

template<class T, class U>
constexpr auto compare_3way_type_t<T, U> compare_3way(const T& a, const U& b);

// ...
}

```

Add a new specification for `compare_3way_type` at the beginning of 23.7.11 [alg.3way]:

The behavior of a program that adds specializations for the `compare_3way_type` template defined in this subclause is undefined.

For the `compare_3way_type` type trait applied to the types `T` and `U`, the member typedef `type` shall be either defined or not present as follows. Let `t` and `u` denote lvalues of types `const remove_reference_t<T>` and `const remove_reference_t<U>`.

- If the expression `t <=> u` is well formed, the member *typedef-name* `type` shall equal `decltype(t <=> u)`.
- Otherwise, if the expressions `t == u` and `t < u` are each well-formed and convertible to `bool`, the member *typedef-name* `type` shall equal `strong_ordering`.
- Otherwise, if the expression `t == u` is well-formed and convertible to `bool`, the member *typedef-name* `type` shall equal `strong_equality`.
- Otherwise, there shall be no member `type`.

A program shall not specialize `compare_3way_type`.

Change the return type of `compare_3way` in 23.7.11 [alg.3way]. Its specification is largely repeated from the above, but the repetition is necessary:

```

template<class T, class U> constexpr auto compare_3way_type_t<T, U> compare_3way(const T& a, const U& b);

```

§ 2.2. Interaction with P1186

P1186 proposed to obviate `std::compare_3way()` in favor of having `<=>` itself perform all of this logic. That paper getting adopted does not change the need for having a type trait like this, though it would make the specification much simpler as the library wording could simply defer to the core wording.

Add a new specification for `compare_3way_type` at the beginning of 23.7.11 [alg.3way]:

The behavior of a program that adds specializations for the `compare_3way_type` template defined in this subclause is undefined.

For the `compare_3way_type` type trait applied to the types `T` and `U`, let `t` and `u` denote lvalues of types `const remove_reference_t<T>` and `const remove_reference_t<U>`. If the expression `t <=> u` is well formed, the member *typedef-name* `type` shall equal `decltype(t <=> u)`. Otherwise, there shall be no member `type`.

§ 3. References

- [\[P1186R0\]](#) "When do you actually use <=>?" by Barry Revzin, 2018-10-07

- [\[alg.3way\]](#) Current Working Draft
- [\[revzin.impl\]](#) "Implementing the spaceship operator for optional" by Barry Revzin, 2017-11-16